

Ejercicio de refuerzo - Semana 1

Procesamiento reproducible de telemetría de servidores de IA

Modalidad	Tiempo sugerido	Entrega
Individual	2-3 horas	Proyecto comprimido

Objetivo

Integrar los contenidos de la primera semana en un único pipeline: representar datos con una abstracción simple, procesarlos con NumPy, analizarlos con Pandas y persistir el resultado en Parquet verificando el contrato de los datos.

Escenario

El archivo `data/server_measurements.csv` contiene mediciones de seis servidores que ejecutan cargas de IA. Cada fila corresponde a una medición en un instante determinado.

```
server | timestamp | gpu_utilization | cpu_utilization | memory_gb | power_w  
| temperature_c
```

Preparación del proyecto

1. Descargar el archivo `ejercicio_semana1_student.zip` desde D2L.
2. Descomprimir el archivo y acceder a la carpeta `ejercicio_semana1_student`.
3. Reconstruir el entorno con `uv sync --group dev`.
4. Inicializar Git y mantener al menos dos commits con mensajes que describan cambios concretos.
5. Antes de entregar, ejecutar `git log --oneline > git_history.txt`.

```
uv run pytest -q  
uv run week1-exercise
```

Tarea 1 - Representar las mediciones

Completar la función `build_measurements`. La función debe seleccionar del DataFrame las características `gpu_utilization`, `cpu_utilization` y `memory_gb`, convertirlas a una matriz NumPy y retornar un objeto `ServerMeasurements`. Completar la clase `ServerMeasurements`. Se debe almacenar una matriz NumPy bidimensional y una tupla con los nombres de las características. Validar que el número de columnas coincida con la cantidad de nombres y que los datos sean numéricos. Implementar también `__repr__`.

Utilizar como características: `gpu_utilization`, `cpu_utilization` y `memory_gb`.

```
Resultado esperado:  
ServerMeasurements(shape=(300, 3), features=('gpu_utilization',  
'cpu_utilization', 'memory_gb'))
```

Tarea 2 - Normalizar y calcular un indicador de carga

Calcular media y desviación estándar por columna. Normalizar las tres características mediante Z-score. Si una desviación estándar es cero, evitar la división por cero.

Después calcular `load\_score` usando los pesos `(0.50, 0.30, 0.20)` y operaciones vectorizadas. No recorrer las observaciones con un `for`.

Resultado esperado:

```
means: [57.99313333 50.54163333 29.45533333]
zscores.shape -> (300, 3)
load_score.shape -> (300,)
```

Tarea 3 - Identificar observaciones para revisión

Crear un nuevo DataFrame a partir del original y agregar las columnas `load\_score` y `requires\_review`. Una fila requiere revisión si `load\_score > 1.5` o `temperature\_c > 80`. No modificar el DataFrame original.

Resultado esperado:

	server	timestamp	load_score	temperature_c	requires_review
23	AI-SRV-01	2026-08-24 09:55:00	3.136218	66.97	True

Tarea 4 - Construir un resumen por servidor

Agrupar las observaciones por `server` y construir una tabla con una fila por servidor. Se debe incluir exactamente las siguientes columnas: `server`, `observations`, `mean\_power\_w`, `max\_temperature\_c`, `mean\_load` y `review\_count`. Considerar que `review\_count` indica cuántas observaciones del servidor tienen `requires\_review=True`.

Resultado esperado:

	server	observations	mean_power_w	max_temperature_c	mean_load	review_count
23	AI-SRV-01	50	363.4598	86.50	-0.614527	2

Tarea 5 - Guardar y verificar el resultado

Guardar el DataFrame analizado en `output/server\_analysis.parquet` mediante PyArrow/Parquet. Volver a leer el archivo y verificar, como mínimo, número de filas, nombres de columnas, esquema y valores (filas antes == filas después, columnas antes == columnas después, esquema antes == esquema después)

Resultado esperado:

```
Round trip verificado: output/server_analysis.parquet
Filas: True
Columnas: True
Esquema: True
Valores: True
```

Verificación con `tests/test\_public.py`

El archivo `tests/test\_public.py` contiene pruebas públicas para verificar cada tarea. Ejecutar los comandos desde la raíz del proyecto, después de reconstruir el entorno con `uv sync --group dev`. No es necesario modificar `test\_public.py`: se debe usar como mecanismo de verificación.

Tarea	Comando	Qué verifica	Salida esperada
1	<code>uv run pytest -v -k "batch or feature_count or non_numeric"</code>	Contrato de ServerMeasurements, entradas inválidas, nombres de características, datos no numéricos y <code>__repr__</code> .	5 passed, 4 deselected
2	<code>uv run pytest -v -k "load_score"</code>	Formas, Z-score, valores finitos y cálculo de <code>load_score</code> .	1 passed, 8 deselected
3	<code>uv run pytest -v -k "enriched_dataframe"</code>	Nuevas columnas, regla de revisión y preservación del DataFrame original.	1 passed, 8 deselected
4	<code>uv run pytest -v -k "summary"</code>	Estructura y columnas del resumen por servidor.	1 passed, 8 deselected
5	<code>uv run pytest -v -k "parquet"</code>	Round trip Parquet: archivo, filas, columnas, esquema y valores.	1 passed, 8 deselected

Al completar todas las tareas, ejecute `uv run pytest -v`. La salida final esperada es `9 passed` (el tiempo de ejecución puede variar). Si aparece `FAILED`, revise el nombre de la prueba y el traceback antes de continuar con la siguiente tarea.

Entregable

Comprimir el proyecto y subirlo a D2L a la carpeta Ejercicios 1 que estará activa hasta las 18h30 del martes 25 de agosto de 2026. La carpeta debe contener el código fuente, `pyproject.toml`, `uv.lock`, `git_history.txt`, el dataset entregado y el archivo Parquet generado. No incluir `.venv/`, cachés ni archivos temporales.